

Profiling from Zero

A Complete Hands-On Guide to perf, Nsight, gprof,
Valgrind, VTune, and beyond

Prepared for CK | IIT Delhi | Nanopore Project
Prof. Kolin Paul | Summer 2025

Batch 3 · GPU Profiling with Nsight · Chapters 11 to 15

Contents

Chapter 11 GPU Architecture Review: What Nsight Actually Measures

Chapter 12 Nsight Systems: Timeline and System-Level Profiling

Chapter 13 Nsight Compute: Kernel-Level Deep Dive

Chapter 14 Nsight Compute: Occupancy, Bank Conflicts, and Warp Stalls

Chapter 15 Nsight: Streams, Async, and Multi-GPU

This batch assumes Chapters 1 to 10: the full perf toolkit, the Valgrind suite (Memcheck, Cachegrind, Callgrind, Massif, Helgrind), gprof, and the five benchmark programs.

CHAPTER 11

GPU Architecture Review

We cross the PCIe bus. Everything you learned about the CPU still applies in spirit, but almost nothing applies in detail. The CPU is a small number of very clever cores that hide latency by being smart: out-of-order execution, branch prediction, deep speculation, large per-core caches. The GPU is the opposite design philosophy taken to an extreme. It is thousands of simple cores that hide latency not by being clever but by having so much work queued that whenever one group of threads stalls, another is ready to run. The CPU hides latency with intelligence. The GPU hides latency with sheer parallelism. If you carry your CPU intuitions onto the GPU unexamined, you will misread every Nsight number you see. So before we touch Nsight, we do for the GPU exactly what Chapter 2 did for the CPU: build the mental model of the machine, so the profiler's numbers become sentences instead of noise.

This matters acutely for our project because Dorado runs entirely on the GPU. Its Transformer attention blocks, its convolutional stem, its CTC decoder are all CUDA kernels. `perf` and `Valgrind`, the entire Batch 1 and Batch 2 toolkit, are blind to all of it; they see only the host-side orchestration. To profile the actual basecalling work we need Nsight, and to read Nsight we need this chapter. Our concrete vehicle is the `matmul_cuda` benchmark, the two-kernel program with a naive global-memory `matmul` and a tiled shared-memory `matmul`, which exercises exactly the bottlenecks that matter for Dorado.

11.1 The GPU Memory Hierarchy

Just as the CPU story was dominated by the memory hierarchy, so is the GPU story, but the hierarchy has different layers, different sizes, and dramatically different latencies. Let me lay it out from fastest to slowest, the GPU analogue of the table in Chapter 2.

- **Registers:** per-thread private storage, roughly 256 KB total per SM split among all resident threads, about 1 cycle latency. This is where a thread's working variables live. Registers are the GPU's most precious resource, and as we will see, running out of them limits how many threads can run at once.
- **Shared memory / L1:** per-SM, configurable, roughly 64 to 164 KB, about 20 to 30 cycle latency. Shared memory is explicitly managed scratchpad that all threads in a block can read and write. It is the single most important optimization lever on the GPU, the analogue of cache blocking on the CPU but under your direct control.
- **L2 cache:** shared across all SMs, a few megabytes, roughly 200 cycle latency. This is the last cache before the slow global memory. Every SM's misses funnel through here.
- **Global memory (VRAM / HBM or GDDR):** off-chip device memory, gigabytes, roughly 400 to 800 cycle latency. This is where your big arrays live. It is fast in bandwidth (hundreds of GB/s) but slow in latency, and that latency is the central thing the GPU architecture is built to hide.

- **Host memory (CPU RAM over PCIe):** the slowest path, reached only by explicit transfers across the PCIe bus, with latency in the thousands of cycles and bandwidth an order of magnitude below device memory. Every byte that crosses this bus is expensive, which is why minimizing host-device transfer is a recurring optimization theme.

Notice the shape difference from the CPU. On the CPU, the caches are automatic and hidden; you influence them only indirectly through access patterns. On the GPU, the most important fast memory, shared memory, is *explicitly managed*. You decide what goes in it and when. This is why GPU optimization feels more hands-on: you are manually staging data into fast memory the way the `matmul_tiled_gpu` kernel cooperatively loads tiles into shared memory before computing on them.

The Central Analogy

A CPU core is a single expert craftsman who works fast and never sits idle because he plans ahead. A GPU SM is a factory floor with hundreds of workers, each slower and dumber, where the trick is that whenever one worker is waiting for a part to arrive from the warehouse (global memory), the floor manager (the warp scheduler) instantly hands work to a different worker who already has their parts. No single worker is impressive. The floor never stops. That is how the GPU turns enormous memory latency into high throughput: not by making any access fast, but by always having other work ready.

11.2 SMs, Warps, and the Execution Model

A GPU is built from Streaming Multiprocessors, the SMs. An SM is the rough equivalent of a CPU core, except that instead of running one or two threads it runs hundreds simultaneously. The number of SMs is how GPUs scale: a small inference GPU like the T4 has 40 SMs, a data-center A100 has 108, a modest laptop GPU like a GTX 1650 has 14. More SMs means more parallel work in flight. When you launch a CUDA kernel, the thousands of thread blocks you launch get distributed across the available SMs, and each SM chews through the blocks assigned to it.

Within an SM, threads do not execute individually. They execute in groups of 32 called **warps**. This is the single most important architectural fact about GPUs, and it has consequences everywhere. All 32 threads in a warp execute the same instruction at the same time, in lockstep, on different data. This is called SIMT, single instruction multiple threads. The hardware fetches one instruction and applies it across all 32 lanes of the warp simultaneously.

The warp is the unit of scheduling. An SM holds many warps resident at once, perhaps 32 or 64 of them, and the SM's warp schedulers pick a ready warp each cycle and issue its next instruction. The instant a warp stalls, waiting for a global-memory load to return, say, the scheduler switches to another warp that is ready. With enough resident warps, there is always one ready, so the SM's execution units stay busy even though every individual memory access takes hundreds of cycles. This is latency hiding through parallelism, and it is the entire reason the GPU tolerates 400-cycle global-memory latency without grinding to a halt.

Why warps explain so much

Two crucial behaviors fall directly out of the warp concept, and you will see both in Nsight.

First, **warp divergence**. Because all 32 threads execute the same instruction, what happens at a branch where some threads go one way and some the other? The hardware cannot run two instructions at once on one warp. So it runs both paths serially, masking off the threads that should not be active on each path. A warp that diverges into two branches takes twice as long, because both halves run while half the lanes sit idle. Divergent branching is a GPU-specific performance killer with no CPU analogue, and Nsight reports it directly as branch efficiency.

Second, **memory coalescing**. When a warp issues a memory load, the hardware looks at the 32 addresses the 32 threads want. If those addresses are contiguous, falling within one or two aligned 128-byte segments, the hardware satisfies them in a single combined memory transaction. This is a coalesced access, and it is maximally efficient. If the 32 addresses are scattered, the hardware must issue many separate transactions, one or more per thread, wasting most of the bandwidth. Look at the naive `matmul_cuda` kernel: threads in a warp have consecutive `col` values, so the access to `B[k*n + col]` is contiguous across the warp and coalesces beautifully. That single design choice is why even the naive GPU kernel is not catastrophic the way naive CPU `matmul` was. Coalescing is the GPU analogue of the cache-line spatial locality from Chapter 2, but it operates per-warp and per-instruction rather than per-access.

How This Connects to Your Project

Dorado's Transformer attention is built from large matrix multiplications and softmax operations, exactly the kind of work that lives or dies by coalescing and divergence. The attention score computation is a batched `matmul` whose memory access pattern determines whether the warps coalesce. The softmax involves reductions that can cause divergence if written naively. When Nsight Compute later tells you Dorado's attention kernel has, say, 60 percent memory throughput and some uncoalesced accesses, you will know precisely what those numbers mean because of this section: the warps are not perfectly contiguous in their global-memory access, leaving bandwidth on the table.

11.3 The Three GPU Bottlenecks

On the CPU, the diagnostic compass was IPC plus cache miss rate, separating compute-bound from memory-bound. The GPU has a three-way version of this, and Nsight Compute's headline analysis (the Speed Of Light section we meet in Chapter 13) is built to tell you which of the three you are in. Learn these three now and the entire Nsight Compute output organizes itself around them.

Bottleneck 1: Compute bound

The SMs are busy nearly every cycle doing arithmetic. The execution units, the floating-point pipes, the tensor cores, are saturated. There is plenty of work and the warps rarely stall. This is the good problem to have, the GPU equivalent of high IPC. If you are compute-bound, the only ways forward are doing less arithmetic, using a faster arithmetic path (tensor cores instead of regular FP, lower precision like FP16 instead of FP32), or accepting that you are near the ceiling. Nsight signals this with high SM utilization and low memory throughput relative to peak.

Bottleneck 2: Memory bandwidth bound

The SMs spend their cycles waiting for data to arrive from global memory. The memory system is moving bytes as fast as it physically can, at or near peak bandwidth, but the kernel demands data faster than the memory can supply it. This is the GPU analogue of the memory-bound CPU case, and it is extremely common because, as we will see, modern GPUs have so much compute that almost everything is memory-bound unless arithmetic intensity is very high. Nsight signals this with memory throughput near peak and SM compute utilization well below peak. The fix is to reduce global-memory traffic, which is exactly what shared-memory tiling does: it stages data into fast on-chip memory so the same bytes are reused many times rather than re-fetched from global memory.

Bottleneck 3: Latency bound

This one has no clean CPU analogue and trips up everyone. The kernel is neither saturating the compute units nor the memory bandwidth. Both are underused. The problem is that there are not enough warps resident on the SM to hide the latency of the operations that are happening. Remember, the GPU hides a 400-cycle memory latency by switching to another ready warp. But if the SM has only a few warps resident, then when they all stall there is nothing to switch to, and the SM sits idle waiting. The execution units are not busy and the memory is not busy; everyone is just waiting. The fraction of the maximum possible resident warps that you actually achieve is called **occupancy**, and low occupancy is the usual cause of being latency bound. Nsight signals this with both compute and memory utilization low and reports occupancy explicitly. The fix is to raise occupancy: use fewer registers per thread, use less shared memory per block, or restructure so more warps fit. Chapter 14 is largely about this.

Key Insight

The CPU question was binary: compute-bound or memory-bound. The GPU question is three-way: compute-bound (SMs saturated, good), memory-bound (bandwidth saturated, reduce traffic), or latency-bound (nothing saturated, raise occupancy). Nsight Compute's Speed Of Light section answers this three-way question in its first two numbers, SM throughput and memory throughput. High one of them tells you which resource binds you. Both low means you are latency-bound and need more warps in flight. Internalize this triage and Nsight stops being intimidating.

11.4 The Key GPU Metrics

Nsight reports dozens of metrics. Here are the ones that carry the most signal, grouped by what they tell you, so that when you see them in Chapters 12 through 15 you already know what they mean.

- **SM utilization / SM throughput:** what fraction of peak the compute units achieved. High means compute-bound or at least compute-busy.
- **Memory throughput / DRAM throughput:** what fraction of peak memory bandwidth you achieved, often quoted in GB/s. High means memory-bound.

- **Achieved occupancy:** the ratio of average active warps to the hardware maximum. Low occupancy is the signature of latency-bound kernels.
- **L2 hit rate:** the fraction of global-memory accesses caught by the L2 cache rather than going all the way to DRAM. Higher is better; tiling raises it.
- **Shared memory efficiency / bank conflicts:** whether your shared-memory accesses are conflict-free or serialized. Chapter 14 covers the 32-bank structure in detail.
- **Warp stall reasons:** a breakdown of why warps were not issuing instructions, each cycle they stalled. The big ones are long-scoreboard (waiting for global memory), barrier (waiting at __syncthreads), and math-pipe-throttle (too many in-flight math ops). These are the GPU's equivalent of the CPU's stalled-cycles counter, but far more diagnostic.

Among the warp stall reasons, learn these names now because they are the most actionable single piece of Nsight Compute output. *Long scoreboard* means the warp is waiting for a global-memory load to complete, the signature of memory-bound or latency-bound behavior. *Short scoreboard* means waiting for a shared-memory access, often a sign of bank conflicts. *Barrier* means waiting at a __syncthreads, the cost of synchronization between cooperative threads. *Math pipe throttle* means the arithmetic units are backed up, a sign of compute-bound. When Nsight tells you which stall reason dominates, it is telling you the bottleneck almost directly.

11.5 The GPU Roofline

The Roofline Model from Chapter 5 transfers to the GPU unchanged in form, only the numbers differ, and it remains the single best framework for answering how-much-faster on the GPU. The two ceilings are still peak compute and peak memory bandwidth, and arithmetic intensity, FLOPs per byte of global-memory traffic, still decides which ceiling binds you. The crucial GPU-specific twist is where the ridge point sits.

Take the NVIDIA T4, a common inference GPU and a likely target for profiling Dorado. Its headline specs are roughly 65 TFLOP/s of FP16 tensor-core throughput, about 8 TFLOP/s of FP32, and 320 GB/s of memory bandwidth. Compute the FP16 ridge point: 65e12 FLOP/s divided by 320e9 byte/s gives about 203 FLOPs per byte. That ridge is enormous. It means that to be compute-bound on the T4's tensor cores, a kernel must do more than 200 floating-point operations for every byte it reads from global memory. Very few kernels achieve that. The consequence is profound and worth stating plainly: on modern GPUs, the tensor cores are so fast that almost everything is memory-bandwidth-bound unless arithmetic intensity is extraordinarily high. This is the GPU version of the lesson from Chapter 5, amplified.

Our `matmul_cuda` benchmark illustrates it perfectly. The naive kernel reads A and B from global memory for every output element with no reuse, so its arithmetic intensity is low, far left of the ridge, deep in memory-bound territory. The tiled kernel stages tiles into shared memory and reuses each loaded byte across an entire tile of computation, raising its effective arithmetic intensity by the tile factor and pushing its dot rightward toward the ridge. The whole point of tiling is to climb the roofline by raising arithmetic intensity, exactly as loop reordering and blocking did on the CPU. Nsight Compute will draw this roofline for you and place both kernels on it, which we will see in

Chapter 13.

How This Connects to Your Project

For the Dorado report page, the GPU roofline is what justifies any claim about how good the current performance is. Nsight Compute has a built-in roofline section that places each kernel as a dot under the T4 roof. The report can then say, for the dominant attention kernel, something like "the kernel achieves X percent of T4 peak memory bandwidth and sits left of the ridge at arithmetic intensity Y, confirming it is memory-bandwidth-bound, so the right optimization is to reduce global-memory traffic, not to add compute."

That sentence is the difference between a report that says Dorado is slow and a report that says exactly which hardware ceiling binds it and which direction to push. The Signal-to-Base caching idea is precisely a traffic-reduction optimization, so the roofline is what motivates it: a memory-bound kernel is helped by caching and fusion, never by more arithmetic. The roofline turns a vague optimization instinct into a measured, defensible recommendation.

Common Pitfall

Carrying CPU intuitions onto the GPU. The GPU does not hide latency with cleverness; it hides it with thousands of warps. Low occupancy on the GPU has no CPU equivalent and is invisible if you think in CPU terms.

Assuming high arithmetic means compute-bound. On a GPU with a ridge point above 200 FLOPs/byte, even fairly arithmetic-heavy kernels are memory-bound. Always check arithmetic intensity against the ridge, not in absolute terms.

Ignoring coalescing and divergence because they have no CPU analogue. These per-warp effects can halve or worse the effective bandwidth and compute of a kernel, and they are the first things to check when a kernel underperforms its roofline position.

Confusing latency-bound with memory-bound. Memory-bound means the bandwidth is saturated; latency-bound means nothing is saturated and you simply lack enough warps. The fixes are opposite: reduce traffic versus raise occupancy.

What We Just Learned

The GPU hides enormous memory latency not with CPU-style cleverness but with massive parallelism: thousands of warps, so that whenever one stalls another is ready. This single idea explains the whole architecture.

The memory hierarchy runs registers, shared memory / L1, L2, global memory (VRAM), and host memory over PCIe. Unlike the CPU, the most important fast memory, shared memory, is explicitly managed by your code.

Threads execute in warps of 32 in lockstep (SIMT). This explains warp divergence (branches that split a warp run serially) and memory coalescing (contiguous per-warp addresses combine into one transaction), neither of which has a clean CPU analogue.

The three GPU bottlenecks are compute-bound (SMs saturated), memory-bandwidth-bound (bandwidth saturated, reduce traffic), and latency-bound (nothing saturated, raise occupancy). Nsight's Speed Of Light triages this in two numbers.

Key metrics: SM throughput, memory throughput, achieved occupancy, L2 hit rate, bank conflicts, and warp stall reasons. Long-scoreboard stalls mean waiting on global memory; barrier stalls mean __syncthreads; math-pipe-throttle means compute-bound.

The roofline transfers to the GPU with a very high ridge point (above 200 FLOPs/byte on T4 tensor cores), so almost everything is memory-bound. Tiling raises arithmetic intensity to climb the roofline, just as blocking did on the CPU.

Before You Move On

You now have the GPU mental model the way Chapter 2 gave you the CPU one. Make sure you can state the three bottlenecks, explain what a warp is and why it causes divergence and coalescing, and explain why low occupancy makes a kernel latency-bound.

Next chapter is Nsight Systems, the GPU equivalent of getting the big-picture timeline: which kernels ran when, how long the host-to-device transfers took, where the gaps and synchronization stalls are. It is the GPU's wide-angle survey, the tool you reach for first, before zooming into any single kernel with Nsight Compute. We profile `matmul_cuda` and set up the exact command for profiling Dorado's pipeline.

CHAPTER 12

Nsight Systems: The Timeline

When you profile a CPU program you start with `perf record` to find the hot function. When you profile a GPU program you start with Nsight Systems to see the whole timeline. The reason is the same in both cases: before you zoom in on any one thing, you need the big picture of where time actually goes. And on the GPU the big picture is full of surprises, because a GPU program is not just GPU work. It is a dance between the CPU and the GPU, with the CPU launching kernels, copying data across PCIe, waiting for results, and launching more. Very often the GPU kernels themselves are fast and the program is slow because of everything around them: transfer overhead, synchronization stalls, gaps where the GPU sits idle waiting for the CPU. Nsight Systems is the tool that shows you this dance as a timeline, and it is almost always where the first real insight comes from.

Let me name the wrong instinct, because it is universal among people new to GPU profiling. Your first instinct is to assume the GPU kernel is the bottleneck and to dive straight into optimizing it. But on a huge fraction of real GPU programs, the kernels are fine and the actual problem is that the GPU is idle half the time, starved by slow host-device transfers or by the CPU failing to keep enough work queued. If you optimize the kernel without looking at the timeline first, you make the fast part faster and the program barely improves, exactly the radix-sort mistake from Chapter 1, now in GPU form. Nsight Systems exists to stop you from making that mistake. Look at the timeline first. Always.

12.1 What Nsight Systems Shows

Nsight Systems, invoked on the command line as `nsys`, captures a unified timeline of your entire application across both the CPU and the GPU. Think of it as a Gantt chart of everything that happened, with several parallel rows of activity aligned on a common time axis.

- **CPU rows:** what each CPU thread was doing, including CUDA API calls like `cudaMalloc`, `cudaMemcpy`, and kernel launches. You see when the CPU issued each command and how long the API call itself took.
- **GPU rows:** what the GPU was actually executing, kernel by kernel, on its timeline. Each kernel appears as a labeled bar whose width is its duration.
- **Memory transfer rows:** the host-to-device and device-to-host copies over PCIe, shown as bars so you can see exactly how much time is spent moving data versus computing on it.
- **Synchronization markers:** where the CPU blocked waiting for the GPU (`cudaDeviceSynchronize`, `cudaEventSynchronize`) and where the GPU sat idle waiting for the next launch.

The power of this view is that it makes the gaps visible. A gap on the GPU row means the GPU was idle. A wide transfer bar means PCIe is the bottleneck. Kernel bars packed back to back with

no gaps mean the GPU is well fed. You read a Nsight Systems timeline the way a doctor reads an EKG: the rhythm and the gaps tell the story before you read any single number.

12.2 Basic Usage

The simplest capture profiles the whole run and writes a report file you open in the Nsight Systems GUI.

```
nvcc -O3 -lineinfo -o matmul_cuda matmul_cuda.cu
nsys profile \
  --trace=cuda,nvtx,osrt \
  --output=matmul_cuda \
  ./matmul_cuda
# produces matmul_cuda.nsys-rep – open it in the Nsight Systems GUI
```

The `--trace` flag selects what to record. *cuda* captures CUDA API calls, kernel executions, and memory transfers, the core of what you want. *nvtx* captures your own custom timeline annotations, which we cover in Section 12.4. *osrt* captures operating-system runtime calls like thread synchronization and I/O, useful for seeing when the CPU is blocked in the OS rather than doing useful work. The `--output` names the report file. The result is a *.nsys-rep* file you open in the GUI to see the timeline.

The `-lineinfo` flag at compile time is the GPU analogue of the CPU's `-g`: it embeds source-line information so the profiler can map GPU activity back to your source. Use it for any profilable build. It does not slow the kernel the way the CPU's `-pg` did; it only adds debug metadata.

12.3 Reading the Timeline

Open the report and you are looking at the aligned rows. For *matmul_cuda* the story is short and clear, which makes it a perfect teaching example. Reading left to right in execution order you will see: a *cudaMalloc* burst as the three device arrays are allocated, then two host-to-device transfer bars as A and B are copied to the GPU, then a thin bar for the naive kernel, then a thin bar for the tiled kernel, then a device-to-host transfer bar copying C back. The shape of this timeline immediately tells you the proportions: how much time was transfer versus compute, and whether the kernels are back to back or separated by gaps.

Here is the first lesson the timeline teaches on this tiny program. For a 1024-square *matmul*, the host-device transfers move 4 MB each way, and on a PCIe link of maybe 12 GB/s effective that is a few hundred microseconds. The kernels themselves run in a comparable time. So a startling fraction of the total time is data movement, not computation. If this were a real workload you ran once, the transfer would be pure overhead. This is the classic GPU lesson made concrete: for small or one-shot work, the cost of getting data onto the GPU and results back off can dwarf the computation, and no amount of kernel optimization helps. The timeline shows this at a glance where no single number would.

What to look for, as a checklist

- **Gaps between kernel launches:** idle GPU time, usually caused by the CPU not queuing the next kernel fast enough or by a synchronization barrier. Each gap is wasted GPU.
- **Transfer time versus kernel time:** if the PCIe bars are wide relative to the kernel bars, you are transfer-bound, and the fix is to transfer less, transfer once, or overlap transfer with compute (Chapter 15).
- **Sequential versus overlapping streams:** by default everything runs in one stream, strictly serialized. If you see transfers and kernels neatly non-overlapping, there is opportunity to overlap them with multiple streams.
- **Synchronization stalls:** long `cudaDeviceSynchronize` bars on the CPU row mean the CPU is blocked waiting for the GPU, which is sometimes necessary and sometimes a sign of an avoidable serialization.

Key Insight

Nsight Systems answers a question Nsight Compute cannot even ask: is my GPU busy at all? A kernel can be perfectly optimized internally and still leave the GPU idle ninety percent of the time because of transfer overhead or poor overlap. The timeline reveals utilization gaps that per-kernel analysis is blind to. This is exactly why you profile with Nsight Systems first and only then zoom into a specific kernel with Nsight Compute. Survey before microscope.

12.4 NVTX: Annotating Your Own Timeline

The raw timeline labels kernels by their function names and shows API calls, but it has no idea about the logical structure of your program. It does not know that these three kernels together form the attention block, or that this region is data loading and that region is the decode step. NVTX, the NVIDIA Tools Extension, lets you annotate your own timeline with named ranges that show up as labeled bars in Nsight Systems. This transforms an undifferentiated wall of kernels into a readable map of your program's phases.

```
#include <nvtx3/nvToolsExt.h>

// wrap a logical region in a named NVTX range
nvtxRangePush("data_transfer_H2D");
cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
nvtxRangePop();

nvtxRangePush("naive_kernel");
matmul_naive_gpu<<<grid, block>>>(d_A, d_B, d_C, N);
cudaDeviceSynchronize();
nvtxRangePop();

nvtxRangePush("tiled_kernel");
matmul_tiled_gpu<<<grid, block>>>(d_A, d_B, d_C, N);
cudaDeviceSynchronize();
nvtxRangePop();
```

C

Each *nvtxRangePush* opens a named range and each *nvtxRangePop* closes it. In the Nsight Systems timeline these appear as a dedicated NVTX row with labeled bars spanning exactly the regions you marked, nested if you nest the pushes. Now instead of staring at anonymous kernel bars you see named phases: data transfer, naive kernel, tiled kernel. For a real program with dozens of kernels this is the difference between a legible timeline and an inscrutable one. NVTX ranges cost almost nothing and are the single best habit for making GPU profiles readable.

12.5 Command-Line Reports Without the GUI

The GUI is excellent but you cannot always use it, especially on a headless cluster node like LUNA where you are working over SSH with no display. Nsight Systems can generate summary statistics directly on the command line from a captured report, which is how you extract numbers for the report without ever opening a window.

```
# overall summary of the captured report
nsys stats matmul_cuda.nsys-rep

# just the GPU kernel time summary, the most useful single table
nsys stats --report cuda_gpu_kern_sum matmul_cuda.nsys-rep

# the memory-transfer summary
nsys stats --report cuda_gpu_mem_time_sum matmul_cuda.nsys-rep
```

bash

The *cuda_gpu_kern_sum* report is the one you will use most. It prints a table of every kernel that ran, with total time, number of instances, average time, and the percentage of total GPU time each kernel consumed. This is the GPU equivalent of perf report's flat profile: a ranked list of where GPU time went, by kernel. For *matmul_cuda* it shows the naive and tiled kernels with their respective times, immediately telling you the speedup tiling achieved. For Dorado it will show the attention kernels, the convolution kernels, and the decode kernels ranked by time, telling you which to investigate with Nsight Compute.

Time(%)	Total Time(ns)	Instances	Avg(ns)	Name	output
71.4	4,812,160	1	4,812,160	matmul_naive_gpu(...)	
28.6	1,928,704	1	1,928,704	matmul_tiled_gpu(...)	

Read this exactly as you read perf report. The naive kernel took 4.81 ms and 71.4 percent of GPU kernel time; the tiled kernel took 1.93 ms and 28.6 percent. The tiled kernel is about 2.5 times faster for the same computation, the GPU echo of the CPU loop-reordering story, and the reason is the same in spirit: tiling reused data from fast on-chip memory instead of re-fetching it from slow global memory. The exact ratio depends on the GPU, but the direction is always this way. This table, generated with one command and no GUI, is precisely what goes into the Dorado report page as the kernel ranking.

How This Connects to Your Project

This is the exact command for profiling Dorado, and the timeline it produces is page one of the report. Run `nsys profile with cuda and nvtx tracing` on a representative basecalling job, then `nsys stats --report cuda_gpu_kern_sum` to get the ranked kernel table. The kernels named something like attention, gemm, conv, and the CTC decode will sort out, and the dominant one is your optimization target.

The command from the project knowledge base is essentially: `nsys profile --trace=cuda,nvtx dorado basecaller hac signal.pod5 --output-dir results/`. The resulting timeline shows the H2D transfers of raw signal, the convolutional stem, the stack of Transformer blocks, and the CTC decode, in execution order. If the GPU rows show gaps, Dorado is being starved by data loading and the fix is on the host side. If the kernel bars are packed and one kernel dominates, that kernel is the target for Nsight Compute in the next chapter. Either way, the timeline tells you where to look before you spend any time looking.

Common Pitfall

Diving into kernel optimization before looking at the timeline. The kernels are often fine and the real problem is GPU idle time from transfers or poor overlap. Survey with Nsight Systems first.

Profiling without NVTX annotations on a complex program and then being unable to tell which kernels belong to which logical phase. Annotate your phases; it costs nothing and saves the analysis.

Forgetting that the default single stream serializes everything. If the timeline shows transfers and kernels never overlapping, that is the default behavior, not a law of physics, and Chapter 15 shows how to overlap them.

Profiling a tiny or one-shot workload and concluding transfers dominate, when for a sustained real workload the transfer is amortized across many kernels. Profile a representative workload, not a toy run, when the conclusion matters for the report.

What We Just Learned

Nsight Systems (nsys) captures a unified CPU-plus-GPU timeline: CPU threads and CUDA API calls, GPU kernel executions, PCIe transfers, and synchronization. It is the GPU survey tool you reach for first.

The timeline makes idle time visible. A kernel can be internally optimal yet leave the GPU idle most of the time due to transfer overhead or poor overlap, which only a timeline reveals.

Basic capture is `nsys profile --trace=cuda,nvtx,osrt --output=name ./prog`, compiled with `-lineinfo` for source mapping. Read the timeline for gaps, transfer-versus-kernel proportion, stream overlap, and synchronization stalls.

NVTX ranges (`nvtxRangePush` / `nvtxRangePop`) annotate your timeline with named logical phases, turning an anonymous wall of kernels into a readable map. They cost almost nothing and are the best habit for legible GPU profiles.

`nsys stats --report cuda_gpu_kern_sum` produces a ranked kernel-time table on the command line with no GUI, the GPU equivalent of `perf report`'s flat profile. This is the kernel ranking that goes on the Dorado report page.

For `matmul_cuda` the tiled kernel beats the naive by roughly 2.5x, the GPU echo of the CPU loop-reordering result, because tiling reuses data from fast shared memory instead of re-fetching from global memory.

Before You Move On

You can now produce the Dorado timeline and the ranked kernel table, which is the top of the report's GPU page and the thing that tells you which kernel to investigate next.

Nsight Systems told you which kernel is hot. It cannot tell you why that kernel is slow inside. For that you need Nsight Compute, which is the next chapter: it zooms into a single kernel and dissects its memory access, occupancy, instruction mix, and bottleneck class, placing it on the roofline and telling you whether it is compute-bound, memory-bound, or latency-bound. Survey done; time for the microscope.

CHAPTER 13

Nsight Compute: Kernel Deep Dive

Nsight Systems told us which kernel is hot. Now we go inside that one kernel and ask why. Nsight Compute is the microscope, the GPU analogue of combining perf stat's hardware counters with Cachegrind's exactness, focused on a single kernel launch. It replays the kernel, reads hundreds of GPU hardware counters, and organizes them into sections that directly answer the three-way bottleneck question from Chapter 11: compute-bound, memory-bound, or latency-bound. If Nsight Systems is where you find the hotspot, Nsight Compute is where you understand it well enough to fix it.

The relationship between the two tools is worth stating once more, plainly, because using the wrong one wastes time. **Nsight Systems** answers "which kernels run, in what order, with what gaps and transfers?" It is timeline, system-level, the whole application. **Nsight Compute** answers "what happens inside this one kernel, cycle by cycle, counter by counter?" It is microarchitectural, single-kernel, deep. You always survey with Systems first, because Compute on the wrong kernel is wasted effort, and because Compute is expensive: to read all those counters it replays the kernel many times, so profiling one kernel in full can take far longer than the kernel itself.

13.1 Basic Usage

Nsight Compute is invoked as *ncu*. The simplest full-detail capture profiles every kernel with the complete set of sections, which is thorough but slow because of the replay.

```
nvcc -O3 -lineinfo -o matmul_cuda matmul_cuda.cu bash

# full detail on every kernel (slow: replays each kernel many times)
ncu --set full ./matmul_cuda

# target one specific kernel by name – far faster and usually what you want
ncu --kernel-name matmul_naive_gpu --set full ./matmul_cuda

# limit to the first launch of a kernel, to avoid profiling thousands of them
ncu --kernel-name matmul_tiled_gpu --launch-count 1 --set full ./matmul_cuda
```

The `--set full` flag requests the complete set of metric sections; without it you get a smaller default set. The `--kernel-name` filter restricts profiling to kernels matching a name, which is essential on a real program where you do not want to replay every kernel, only the one Nsight Systems flagged. The `--launch-count` limits how many launches of a matching kernel to profile, important for Dorado where the same kernel runs thousands of times and you only need a representative sample. Because of the replay overhead, always narrow to the kernel and launch count you care about; running `--set full` on everything in a large program can take a very long time.

The `-lineinfo` flag at compile time again earns its keep here: with it, Nsight Compute can attribute counters to individual source lines in its Source section, the GPU analogue of `perf annotate`. Without it you get kernel-level totals but no per-line detail.

13.2 The Sections of Nsight Compute Output

Nsight Compute organizes its output into named sections, each answering a different question. Learn the five that carry the most signal and the rest fall into place around them. I will go through them in the order you should read them, which is also roughly the order Nsight Compute presents them.

GPU Speed Of Light: the headline triage

This is the section you read first, every time, because it answers the three-way bottleneck question in two numbers. Speed Of Light reports the achieved percentage of two ceilings: **Compute (SM) throughput** and **Memory throughput**, each as a percentage of the hardware peak. These two numbers triage the kernel immediately, exactly as Chapter 11 promised.

- **High SM throughput, lower memory throughput:** compute-bound. The arithmetic units are the limit. Optimize by reducing work or using faster precision.
- **High memory throughput, lower SM throughput:** memory-bandwidth-bound. The bandwidth is the limit. Optimize by reducing global-memory traffic, which means tiling, fusion, caching.
- **Both low:** latency-bound. Neither resource is saturated; you lack enough warps to hide latency. Optimize by raising occupancy.

For the naive `matmul_cuda` kernel, Speed Of Light shows memory throughput high (the kernel hammers global memory reading A and B with no reuse) and SM throughput lower, the unmistakable signature of memory-bandwidth-bound. For the tiled kernel, memory throughput drops (tiling cut the global-memory traffic) and SM throughput rises (the units are busier relative to the reduced memory load), moving the kernel toward compute-bound. That shift, visible in two numbers, is the entire tiling optimization expressed in Nsight Compute's headline section.

Memory Workload Analysis: where the bytes go

This section breaks down the memory system: DRAM throughput in GB/s, the L1 and L2 hit rates, the total bytes moved at each level, and whether your accesses were coalesced. For the naive kernel you will see high DRAM traffic and a modest L2 hit rate, because every output element re-reads A and B from memory. For the tiled kernel the L2 hit rate climbs and DRAM traffic falls, because the staged tiles are reused from shared memory and the remaining global accesses hit L2 more often. The L2 hit rate improvement under tiling is the GPU echo of the CPU's LL miss rate improvement under blocking from Chapter 7: the same data, reused from a closer level instead of refetched from the farthest one.

Compute Workload Analysis: which pipes are busy

This section shows the utilization of the arithmetic pipelines: the FP32 pipe, the FP64 pipe, the FP16 and tensor-core pipes, the integer pipe, the load-store unit. It tells you which kind of work dominates. For a kernel that should be using tensor cores but shows them idle while the FP32 pipe is busy, this section reveals that you are leaving the fast path unused, a common and expensive mistake in deep-learning kernels. For Dorado, this section tells you whether the attention matmuls are actually hitting the tensor cores or falling back to slower FP32, which directly affects how close to the roofline ceiling the kernel can get.

Occupancy: how many warps are resident

This section reports achieved occupancy, the ratio of average active warps to the hardware maximum, and crucially it tells you the limiting factor: are you limited by registers per thread, by shared memory per block, or by block size? This is the diagnostic for latency-bound kernels. If Speed Of Light said both throughputs are low and Occupancy says you are at 25 percent limited by registers, you know the fix: reduce register usage to fit more warps. Chapter 14 is devoted to reading and improving this section, so here it is enough to know it exists and that it is where latency-bound kernels are diagnosed.

Source Counters: per-line attribution

With `-lineinfo`, this section maps counters to individual source lines, showing for each line the instructions executed, the stall reasons, and the memory accesses. It is the GPU analogue of `perf annotate` and `cg_annotate`. For the naive kernel it will pin the global-memory stalls to the inner accumulation loop where A and B are read; for the tiled kernel it will show the stalls shifting to the `__syncthreads` barriers, because once the memory traffic is reduced the synchronization becomes the next most significant cost. Reading the Source section is how you go from "this kernel is memory-bound" to "this specific line is where the memory stalls happen."

Reading Nsight Compute in Order

Read Speed Of Light first; it triages the kernel into compute-bound, memory-bound, or latency-bound in two numbers. If memory-bound, read Memory Workload Analysis to see the traffic and hit rates. If compute-bound, read Compute Workload Analysis to see which pipe. If latency-bound, read Occupancy to find the limiter. Then read the Source section to pin the bottleneck to specific lines. That order, headline then detail then source, turns a hundreds-of-counters dump into a five-minute diagnosis.

13.3 The Worked Example: Both `matmul_cuda` Kernels

Let me walk both kernels through Nsight Compute the way you would for the report, because the contrast is the clearest possible illustration of what the tool reveals. Run each kernel with full detail and read the Speed Of Light and memory sections.

```
ncu --kernel-name matmul_naive_gpu --set full --launch-count 1 \  
--export naive_ncu ./matmul_cuda  
  
ncu --kernel-name matmul_tiled_gpu --set full --launch-count 1 \  
--export tiled_ncu ./matmul_cuda
```

bash

The naive kernel's profile reads like this story. Speed Of Light: memory throughput high, SM throughput moderate, verdict memory-bandwidth-bound. Memory Workload Analysis: high DRAM traffic, because each of the N^2 output elements drives $2N$ global-memory reads with no reuse, and a modest L2 hit rate. Warp stall reasons (which we explore fully in Chapter 14): dominated by long-scoreboard, meaning warps are constantly waiting for global-memory loads. The diagnosis is unambiguous and matches the roofline: the naive kernel is starved for bandwidth because it re-reads everything from global memory.

The tiled kernel's profile tells the optimization's success story. Speed Of Light: memory throughput noticeably lower, SM throughput higher, the kernel has moved toward compute-bound. Memory Workload Analysis: DRAM traffic reduced by roughly the tile factor, because each tile loaded into shared memory is reused across an entire block of computation, and a markedly higher L2 hit rate. Warp stall reasons: long-scoreboard reduced, with barrier stalls now more prominent because the `__syncthreads` calls between tile phases are the new significant cost. The diagnosis: tiling worked, converting a bandwidth-bound kernel into a more balanced one, and the next optimization frontier is the synchronization, not the memory. That is exactly the kind of before-and-after, with the bottleneck shifting as you fix the first problem, that makes a profile report compelling.

13.4 The Built-In Roofline

Nsight Compute can draw the GPU roofline from Chapter 11 directly, placing the kernel as a dot under the compute and memory ceilings of the actual GPU you ran on. This is the single most powerful visualization in the tool for answering how-much-faster, and it is generated automatically when you profile with the full set.

On the roofline chart, the naive kernel appears as a dot low and to the left, below the slanted memory-bandwidth ceiling, confirming it is memory-bound with headroom to climb if you raise arithmetic intensity. The tiled kernel appears as a dot higher and to the right, closer to the ridge, because tiling raised its arithmetic intensity by reusing data. Seeing both dots on the same roofline is seeing the optimization as a movement along the roof: tiling pushed the kernel up the bandwidth ceiling toward the compute ceiling by reducing the bytes moved per FLOP. The distance from each dot to the roof is the remaining headroom, the precise answer to how much faster the kernel could still go.

And Here Is Where It Gets Beautiful

The CPU roofline in Chapter 5 was something you computed by hand from FLOP counts and bandwidth measurements. Nsight Compute draws the GPU roofline for you, automatically, from the actual hardware counters of the run, with your kernel plotted as a dot at its true measured position. You do not estimate arithmetic intensity; the tool measures it. You do not look up peak bandwidth; the tool knows the GPU. The roofline that took a section of arithmetic on the CPU is a single chart here, and it tells you exactly how far below the ceiling you are and which ceiling you are under. When you need to tell Prof. Paul how good Dorado's performance is, this chart is the answer, measured rather than estimated.

13.5 Nsight Compute on Dorado

Now the project payoff. Nsight Systems told you which Dorado kernel dominates, almost certainly one of the attention kernels in the Transformer stack. Nsight Compute dissects it.

```
# profile the dominant attention kernel, a few launches for a representative sample
ncu --kernel-name "regex:.*attention.*|.gemm.*" \
    --launch-count 3 \
    --set full \
    --export dorado_attention_ncu \
    dorado basecaller hac signal.pod5 --output-dir results/
```

bash

The kernel-name filter accepts a regex, so you can match the attention or gemm kernels without knowing their exact mangled names. The launch count of three captures a representative sample without replaying the kernel thousands of times. What you will likely find, given the T4's very high roofline ridge from Chapter 11, is that the attention kernel is memory-bandwidth-bound: Speed Of Light showing memory throughput dominant, the roofline placing it left of the ridge, warp stalls dominated by long-scoreboard. That is the hypothesis from Chapter 1, now measured. The Memory Workload Analysis will quantify the DRAM throughput as a percentage of the T4's 320 GB/s peak, which is the exact number the report needs.

How This Connects to Your Project

This chapter produces the heart of the Dorado report page: the per-kernel table with duration, SM occupancy, DRAM throughput, and L2 hit rate, plus the roofline placement that classifies the kernel as memory-bound or compute-bound. Run Nsight Compute on the dominant kernel, read Speed Of Light for the classification, read Memory Workload Analysis for the throughput numbers, and read the roofline for the headroom.

Crucially, the classification drives the recommendation. If the attention kernel is memory-bandwidth-bound and sits left of the ridge, the report should recommend a traffic-reduction optimization, never more compute. The Signal-to-Base caching idea is exactly such an optimization: it reduces redundant global-memory traffic by reusing intermediate results, which a memory-bound kernel rewards. Nsight Compute is what proves the kernel is memory-bound and therefore that caching, not more arithmetic, is the right lever. Without this measurement the recommendation would be a guess; with it, it is a defensible conclusion backed by the roofline.

Common Pitfall

Running `ncu --set full` on every kernel of a large program and waiting an enormous time because of the replay overhead. Always narrow to the kernel name and a small launch count.

Skipping Nsight Systems and going straight to Nsight Compute on a guessed kernel. Survey first; Compute on the wrong kernel is wasted replay time.

Forgetting `-lineinfo` and then being unable to read the Source section's per-line attribution. Compile profilable builds with `-lineinfo`.

Reading the sections out of order and drowning in counters. Speed Of Light first for the triage, then the matching detail section, then Source. Headline, detail, source.

What We Just Learned

Nsight Compute (ncu) is the single-kernel microscope: it replays a kernel, reads hundreds of GPU counters, and organizes them into sections. Nsight Systems finds the hot kernel; Nsight Compute explains it. Always survey before microscope.

Narrow with `--kernel-name` and `--launch-count` because the replay overhead makes `--set full` on everything very slow. Compile with `-lineinfo` for per-line Source attribution.

Read the sections in order: Speed Of Light triages compute-bound versus memory-bound versus latency-bound in two numbers; Memory Workload Analysis shows traffic and hit rates; Compute Workload Analysis shows which arithmetic pipe; Occupancy finds the warp limiter; Source pins the bottleneck to lines.

The naive `matmul_cuda` kernel is memory-bandwidth-bound (high memory throughput, long-scoreboard stalls). The tiled kernel shifts toward compute-bound (lower DRAM traffic, higher L2 hit rate, barrier stalls become prominent). The bottleneck moves as you fix the first problem.

Nsight Compute draws the GPU roofline automatically from real counters, plotting the kernel as a measured dot under the ceilings. Tiling moves the dot up and right toward the ridge. The distance to the roof is the remaining headroom.

On Dorado, the dominant attention kernel is likely memory-bandwidth-bound given the T4's high ridge point, which is the measured justification for a traffic-reduction optimization like Signal-to-Base caching rather than more compute.

Before You Move On

You can now classify any kernel as compute-, memory-, or latency-bound and place it on the roofline with real numbers, which is the core of the Dorado report page and the basis for a defensible optimization recommendation.

The next chapter goes deeper into the two areas Nsight Compute flags but that need real understanding to act on: occupancy (how many warps fit, and what limits them, the diagnosis for latency-bound kernels) and the per-warp efficiency issues of bank conflicts and warp stalls. These are the levers you pull once Speed Of Light has told you the kernel is latency-bound or once the Source section shows shared-memory or synchronization stalls. It is the difference between knowing a kernel is slow and knowing exactly which knob to turn.

CHAPTER 14

Occupancy, Bank Conflicts, Warp Stalls

Nsight Compute's Speed Of Light section triages a kernel in two numbers, but two of the three bottleneck classes need real understanding to act on. A latency-bound kernel needs more occupancy, but what controls occupancy and how do you raise it? A kernel stalling on shared memory needs its bank conflicts fixed, but what is a bank conflict? A kernel stalling at synchronization needs its warp stall reasons understood. This chapter is the deep dive into those three intertwined topics: occupancy, bank conflicts, and warp stalls. These are the actual knobs you turn once the headline analysis has pointed you at them. Knowing a kernel is latency-bound is diagnosis; raising its occupancy is treatment, and treatment requires understanding the mechanism.

14.1 Occupancy In Depth

Occupancy is the ratio of warps actually resident on an SM to the maximum the SM can hold. Recall from Chapter 11 why it matters: the GPU hides memory latency by switching among resident warps, so when one stalls another runs. With high occupancy there is always a ready warp and the SM stays busy. With low occupancy, when the few resident warps all stall on memory there is nothing to switch to and the SM idles. Low occupancy is the defining cause of latency-bound kernels. So the question is, what determines how many warps fit, and the answer is a competition over three finite per-SM resources.

The three resources that limit occupancy

An SM has a fixed budget of three things, and a kernel's resident-warp count is limited by whichever runs out first. This is the occupancy calculation, and Nsight Compute's Occupancy section reports exactly which of the three is your limiter.

- **Registers:** the SM has a fixed register file, roughly 65536 32-bit registers. Each thread needs some registers for its variables. If a kernel uses 64 registers per thread, then $65536 / 64 = 1024$ threads, which is 32 warps, can reside. If it uses 128 registers per thread, only 512 threads, 16 warps, fit. Register usage is the most common occupancy limiter, and it is set by the complexity of your kernel: more local variables and more arithmetic means more registers.
- **Shared memory:** the SM has a fixed shared-memory pool, configurable up to roughly 100 to 164 KB. Each block reserves shared memory for its lifetime. If a block uses 48 KB of shared memory and the SM has 96 KB, only two blocks fit at once, capping the resident warps regardless of how many registers are free. Our tiled matmul uses shared memory for its tiles, so its tile size directly trades off against occupancy.
- **Block size and block count:** the SM has a hard cap on resident blocks (often 16 or 32) and on resident warps (often 48 or 64). Very small blocks can hit the block-count cap before using the warp budget; very large blocks can waste capacity. The block size interacts with the other

two limits.

The occupancy calculator combines these: given registers per thread, shared memory per block, and block size, it computes the maximum resident warps as the minimum allowed by all three constraints. Nsight Compute does this automatically and, critically, names the limiting factor. When the Occupancy section says "theoretical occupancy 50 percent, limited by registers," it is telling you that your register usage is what caps the warp count, and that reducing registers is the lever to pull. When it says "limited by shared memory," the lever is your shared-memory usage, perhaps a smaller tile.

Registers as the occupancy killer

Register pressure is the most frequent and most subtle occupancy problem, so let me dwell on it. A kernel with a lot of arithmetic and many live variables needs many registers per thread. The compiler allocates them, and the more it allocates, the fewer warps fit, and the lower your occupancy, and the more latency-bound you become. There is a real tension: a kernel optimized to do more work per thread (more instruction-level parallelism, fewer memory round-trips) often uses more registers and so achieves lower occupancy. Sometimes lower occupancy with more work per thread is faster; sometimes higher occupancy with less work per thread is faster. This is a genuine tradeoff that you resolve by measuring both, which is exactly what Nsight Compute lets you do.

You have two levers to control register usage explicitly when the compiler's choice hurts occupancy.

```
// Lever 1: hard cap registers at compile time (whole-file)
// nvcc -maxrregcount=32 ...
// forces <= 32 registers/thread, raising occupancy, but excess
// variables SPILL to local memory (global memory), which is slow.

// Lever 2: per-kernel hint that the compiler respects
__global__ void __launch_bounds__(256, 4) my_kernel(...) {
    // tells the compiler: target 256 threads/block, at least 4
    // resident blocks/SM. The compiler then limits registers to
    // make that occupancy achievable.
    ...
}
```

C

The `-maxrregcount` flag is the blunt instrument: it caps registers for the whole compilation unit, which raises occupancy but risks register spilling, where variables that no longer fit in registers get stored in local memory, which is actually global memory and therefore slow. A kernel that spills can be slower at high occupancy than it was at low occupancy, because the spill traffic costs more than the occupancy gains. The `__launch_bounds__` qualifier is the precise instrument: it tells the compiler the block size you will launch and the minimum blocks per SM you want resident, and the compiler limits registers per thread to make that occupancy achievable, ideally without spilling. Use `__launch_bounds__` when you know your launch configuration and want the compiler to optimize register allocation for that specific occupancy target.

Key Insight

More occupancy is not always faster. Occupancy is a means to an end: hiding latency. If your kernel already hides its latency with the warps it has, adding more warps does nothing, and forcing more warps by capping registers can hurt if it causes spilling. The right mental model is: raise occupancy only when Nsight Compute says you are latency-bound and the Occupancy section shows you are below the level needed to hide your latency. Chasing 100 percent occupancy for its own sake is a classic novice mistake. Occupancy is a knob, not a goal.

Shared memory and tile size in the tiled kernel

Our tiled matmul makes the shared-memory occupancy tradeoff concrete. The kernel declares two shared arrays of TILE-by-TILE floats. With TILE equal to 16, that is two arrays of 256 floats, 1 KB each per block, 2 KB total, leaving plenty of room for many resident blocks, so shared memory does not limit occupancy. If you raised TILE to 32, the shared arrays become 4 KB each, 8 KB per block, and fewer blocks fit; at TILE equal to 64 they become 16 KB each, 32 KB per block, and the SM might hold only a handful of blocks, dropping occupancy sharply. The larger tile reduces global-memory traffic (more reuse per loaded tile) but raises shared-memory usage (lower occupancy). The sweet spot balances the two, and you find it by profiling several tile sizes and reading both the Memory section and the Occupancy section for each. This is the GPU's version of the cache-blocking tile-size tuning from the CPU chapters, with occupancy as the extra constraint.

14.2 Bank Conflicts In Depth

Shared memory is fast, but it has an internal structure that can serialize accesses if you fight it, and that serialization is the bank conflict. Understanding it is essential for any kernel that uses shared memory, which includes the tiled matmul and most of Dorado's attention kernels.

Shared memory is divided into 32 **banks**, one per warp lane. The bank of a given 4-byte word is determined by its address: bank index equals the word address divided by 4, modulo 32. The hardware can service one access per bank per cycle. So if the 32 threads of a warp access 32 different banks, all 32 accesses happen in a single cycle, the ideal conflict-free case. But if two threads in the warp access different addresses that map to the *same* bank, the hardware cannot service them in one cycle; it serializes them into two cycles. This is a 2-way bank conflict. In the worst case, all 32 threads access different addresses in the same bank, a 32-way conflict, serializing into 32 cycles and cutting shared-memory throughput to one thirty-second of peak.

- **Conflict-free:** 32 threads access 32 different banks (or all access the same address, which broadcasts). One cycle. Ideal.
- **2-way conflict:** two threads hit the same bank at different addresses. Serialized to 2 transactions, half throughput on that access.
- **32-way conflict:** all 32 threads hit the same bank at different addresses. Serialized to 32 transactions, 1/32 throughput. Catastrophic.

The classic cause of bank conflicts is a stride that is a multiple of 32. If threads access shared memory with a stride of 32 elements, every thread lands in the same bank, the worst case. This is why a common fix is **padding**: declaring a shared array one element wider than needed, so that consecutive rows start in different banks and column accesses spread across banks instead of colliding. Declaring the tile as TILE-by-(TILE+1) instead of TILE-by-TILE shifts each row by one bank and eliminates the conflict that a TILE-by-TILE column access would otherwise cause.

Nsight Compute reports bank conflicts directly. The metric to look for is shared-memory bank conflicts, exposed as the counter `l1tex__data_bank_conflicts_pipe_lsu_mem_shared` and summarized in the Memory Workload Analysis section as a shared-memory efficiency percentage. When the Source section shows short-scoreboard stalls (waiting on shared memory) concentrated on a particular shared-memory access, bank conflicts are the likely cause, and the fix is usually padding or restructuring the access pattern.

A Useful Analogy

Picture 32 bank tellers, each serving one customer per minute. Shared memory is fast when your 32 customers each go to a different teller; everyone is served in one minute. A bank conflict is when several of your customers all queue at the same teller while the others stand idle; now it takes as many minutes as the longest queue. Padding the array is like renumbering your customers so they spread out across all 32 tellers instead of bunching at one. The hardware is willing to serve 32 in parallel; bank conflicts are you accidentally making them stand in one line.

14.3 Warp Stall Analysis

We have referenced warp stall reasons in passing; now we treat them properly, because they are the most actionable single output of Nsight Compute. Every cycle a warp is not issuing an instruction, the hardware records why, and Nsight Compute aggregates these into a breakdown of stall reasons. The dominant stall reason tells you the bottleneck almost directly. Here are the ones that matter, what they mean, and what they imply.

- **Long scoreboard (stall_long_sb):** the warp is waiting for a global-memory load to return. This is the signature of memory-bound and latency-bound kernels. It dominates the naive `matmul_cuda` kernel, which constantly reads A and B from global memory. High long-scoreboard means reduce global-memory traffic or raise occupancy to hide the latency.
- **Short scoreboard (stall_short_sb):** the warp is waiting for a shared-memory access. Often a sign of bank conflicts, since conflicts serialize shared accesses and make warps wait longer. High short-scoreboard points you at the bank-conflict analysis above.
- **Barrier (stall_barrier):** the warp is waiting at a `__syncthreads` for the other warps in its block to arrive. This is the cost of cooperative synchronization. It becomes prominent in the tiled kernel once tiling has reduced the memory stalls, because the `__syncthreads` between tile phases is then the next largest cost.
- **Math pipe throttle (stall_math_throttle):** the arithmetic pipelines are backed up with too many in-flight operations. This is a sign of compute-bound behavior, the good problem, and it

means the units are saturated.

- **No instruction (stall_no_instruction):** there is no instruction available to issue, often because instruction-level parallelism is exhausted or because of instruction-cache misses. Less common in numerical kernels.
- **Dispatch stall (stall_dispatch):** the instruction is ready but the dispatch unit is busy. Usually a second-order effect.

The practical method is simple and powerful. Read the warp stall breakdown, find the dominant reason, and let it point you at the fix. Long scoreboard dominant: memory-bound, reduce traffic or raise occupancy. Short scoreboard dominant: shared-memory contention, check bank conflicts. Barrier dominant: synchronization overhead, consider fewer syncs or larger tiles to amortize them. Math throttle dominant: you are compute-bound, which means you are near the ceiling and should consider faster precision or accept the result. The stall breakdown converts the abstract bottleneck class into a concrete reason and therefore a concrete next step.

Watch how this plays out across our two kernels, because it shows the bottleneck migrating as you optimize. The naive kernel's stalls are dominated by long scoreboard: it is starved for global-memory bandwidth. You apply tiling. Now the tiled kernel's long-scoreboard stalls have dropped, because tiling reused data from shared memory and cut the global traffic, but barrier stalls have risen, because the `__syncthreads` calls between tile phases are now the largest remaining cost. The optimization did not eliminate stalling; it moved the stall from memory to synchronization. That migration is the normal pattern of optimization: you fix the biggest bottleneck and the next one becomes visible. Nsight Compute's stall breakdown is what makes the migration legible, so you always know what the current binding constraint is.

How This Connects to Your Project

For Dorado's attention kernels, the warp stall breakdown is the most direct guide to optimization. If long scoreboard dominates, the kernel is memory-bandwidth-bound and the fix is traffic reduction, the Signal-to-Base caching direction. If short scoreboard dominates, there are bank conflicts in the attention's shared-memory usage to fix with padding. If barrier dominates, the synchronization between the attention's cooperative phases is the cost, and fusing operations to reduce syncs would help.

The occupancy analysis matters for Dorado too. If Nsight Compute classifies the attention kernel as latency-bound with low occupancy limited by registers, the lever is register reduction via `__launch_bounds__` to fit more warps and hide the attention's memory latency. The point is that each Nsight Compute diagnosis maps to a specific, named code change. The report's recommendation section is strongest when it says not just "the kernel is memory-bound" but "the kernel shows 70 percent long-scoreboard stalls at 40 percent occupancy, indicating it is bandwidth-bound and would benefit from reducing global-memory traffic," because that sentence names both the evidence and the fix.

Common Pitfall

Chasing 100 percent occupancy as a goal. Occupancy is a means to hide latency. If latency is already hidden, more occupancy does nothing, and forcing it via register caps can cause spilling that makes the kernel slower.

Ignoring bank conflicts because the kernel uses shared memory and shared memory is supposed to be fast. A 32-way conflict makes shared memory thirty-two times slower; padding the array is often a one-line fix.

Reading only Speed Of Light and stopping. The warp stall breakdown is where the abstract bottleneck class becomes a concrete reason and a concrete fix. Always read the stall reasons.

Expecting one optimization to remove all stalling. Optimization migrates the bottleneck: fix memory and synchronization becomes the cost; fix synchronization and something else appears. Re-profile after each change and follow the new dominant stall.

What We Just Learned

Occupancy is the ratio of resident warps to the SM maximum, and it is limited by whichever of three resources runs out first: registers per thread, shared memory per block, or block-size caps. Nsight Compute's Occupancy section names the limiter.

Registers are the most common occupancy limiter. Control them with `-maxrregcount` (blunt, risks spilling to slow local memory) or `__launch_bounds__` (precise, hints the compiler to your target occupancy). More occupancy is not always faster; it only helps when you are latency-bound.

Larger tiles in the tiled matmul reduce global-memory traffic but raise shared-memory usage and lower occupancy. The sweet spot balances the two and is found by profiling several tile sizes.

Shared memory has 32 banks; bank index is word-address modulo 32. Conflict-free access uses 32 different banks in one cycle; a 32-way conflict serializes into 32 cycles. Stride-of-32 access is the classic cause; padding arrays to `TILE+1` is the classic fix.

Warp stall reasons are the most actionable Nsight Compute output. Long scoreboard means waiting on global memory (memory-bound); short scoreboard means shared-memory contention (check bank conflicts); barrier means `__syncthreads` cost; math throttle means compute-bound. The dominant reason names the fix.

Optimization migrates the bottleneck. Tiling shifts the naive kernel's dominant stall from long-scoreboard (memory) to barrier (synchronization). Re-profile after each change and chase the new dominant stall.

Before You Move On

You can now act on every Nsight Compute diagnosis: raise occupancy by managing registers and shared memory, eliminate bank conflicts by padding, and read warp stalls to find the binding constraint. Combined with Chapter 13's section-reading, you can take any kernel from "it is slow" to "here is the specific line and the specific code change."

The final GPU chapter steps back out to the system level for the optimizations that live between kernels rather than inside them: CUDA streams for overlapping transfers with computation, pinned memory that makes async transfers possible, and the timing discipline of `cudaEvent` for measuring GPU work correctly. These are the host-side optimizations that the Nsight Systems timeline revealed as opportunities, and they are how you close the GPU-idle gaps that no amount of in-kernel tuning can fix.

CHAPTER 15

Streams, Async, and Multi-GPU

We close the GPU batch where Nsight Systems pointed us in Chapter 12: the gaps. The timeline showed that even a perfectly tuned kernel can leave the GPU idle, starved while data crawls across PCIe or while the CPU fails to queue the next piece of work. Those gaps are not fixable from inside a kernel. No amount of occupancy tuning or bank-conflict elimination closes a gap where the GPU is simply waiting for the next transfer to finish. The fix lives between the kernels, in how you orchestrate transfers and launches, and the mechanism is CUDA streams. This chapter is about overlapping work so the GPU never sits idle, about the pinned memory that overlap requires, and about timing GPU work correctly so your measurements mean what you think they mean. It is the host-side complement to everything we have done inside kernels.

15.1 The Problem: Serial Transfer and Compute

Recall the `matmul_cuda` timeline from Chapter 12. It ran strictly in order: copy A and B to the device, run the naive kernel, run the tiled kernel, copy C back. Every step waited for the previous one to finish completely. The transfers and the kernels never overlapped. While the H2D transfer ran, the SMs sat idle; while the kernel ran, the PCIe link sat idle. Each resource was used only part of the time, and the total time was the sum of all the parts. This serialization is the default behavior of CUDA, and for a single batch of work it is often fine. But for a pipeline that processes many batches, like Dorado streaming through thousands of signal chunks, this serialization is pure waste: the GPU could be computing on batch N while batch N+1 is being transferred.

Name the wrong instinct here, because it is common. Faced with a transfer-bound timeline, the instinct is to make the transfer faster, perhaps by using a faster bus or compressing the data. Sometimes that helps. But the deeper fix is usually not to make the transfer faster; it is to make the transfer *invisible* by overlapping it with computation. If you can transfer batch N+1 at the same time as you compute batch N, then the transfer time disappears behind the compute time, and the pipeline runs as fast as whichever of the two is slower, not the sum of both. That overlap is what streams enable, and it is the single most impactful host-side optimization for a streaming workload like Dorado.

15.2 CUDA Streams

A CUDA stream is an ordered queue of GPU operations. Operations within a single stream execute in order, one after another. But operations in *different* streams can execute concurrently, overlapping in time, subject to the hardware's ability to run them at once. The default stream, which everything uses unless told otherwise, is a single serializing queue, which is why the default behavior is fully serial. The moment you create multiple streams and distribute work across them, the hardware can overlap operations from different streams: a transfer in stream 1 can run while a kernel in stream 2 computes, because they are independent queues.

The canonical use is double-buffering a pipeline. You split the work into chunks and alternate them across two streams, so that while stream A computes on its chunk, stream B transfers the next chunk, and then they swap. The transfers hide behind the computation. Here is the shape of it.

```
cudaStream_t s0, s1;
cudaStreamCreate(&s0);
cudaStreamCreate(&s1);

for (int chunk = 0; chunk < num_chunks; chunk++) {
    cudaStream_t s = (chunk % 2 == 0) ? s0 : s1;    // alternate streams

    // async copy this chunk H2D in stream s (non-blocking)
    cudaMemcpyAsync(d_in[chunk], h_in[chunk], chunk_bytes,
                   cudaMemcpyHostToDevice, s);

    // launch the kernel on this chunk in the SAME stream s
    process_kernel<<<grid, block, 0, s>>>(d_in[chunk], d_out[chunk]);

    // async copy the result D2H in stream s
    cudaMemcpyAsync(h_out[chunk], d_out[chunk], chunk_bytes,
                   cudaMemcpyDeviceToHost, s);
}
cudaStreamSynchronize(s0);
cudaStreamSynchronize(s1);
```

C

The key is *cudaMemcpyAsync* with a stream argument, which returns immediately instead of blocking, and the kernel launch with the stream as its fourth launch parameter. Because even and odd chunks go to different streams, the hardware can transfer chunk N+1 in one stream while computing chunk N in the other. In the Nsight Systems timeline this shows up beautifully: instead of transfer and kernel bars in a single non-overlapping row, you see two stream rows with their bars staggered and overlapping, the GPU busy throughout. The gaps from Chapter 12 close. That visual transformation, from a sparse serial timeline to a dense overlapping one, is the goal of stream optimization, and Nsight Systems is how you confirm you achieved it.

15.3 Pinned Memory: The Prerequisite for Overlap

There is a catch that traps everyone the first time, and it is invisible until you understand it. The asynchronous overlap above only works if the host memory is **pinned**, also called page-locked. Ordinary host memory allocated with `malloc` is pageable, meaning the operating system can move it around or swap it to disk at will. The GPU's DMA engine, which performs transfers without involving the CPU, cannot safely transfer from memory that might move underneath it. So when you call `cudaMemcpyAsync` on pageable memory, CUDA silently falls back to a synchronous staging copy: it first copies your data to an internal pinned buffer, then transfers from there, and the whole thing blocks. Your async copy is secretly synchronous, and your beautiful overlapping pipeline runs fully serial, and you cannot figure out why the timeline shows no overlap.


```
// pinned (page-locked) allocation – enables true async transfer
float *h_A;
cudaMallocHost(&h_A, bytes);    // pinned: DMA can transfer directly
// ... use h_A ...
cudaFreeHost(h_A);

// versus ordinary pageable allocation:
float *h_B = malloc(bytes);    // pageable: cudaMemcpyAsync silently
                                // falls back to a blocking staged copy

// ... use h_B ...
free(h_B);
```

C

Use *cudaMallocHost* (or *cudaHostAlloc* for more control) to allocate pinned host memory, and free it with *cudaFreeHost*. Pinned memory has two benefits: it enables true asynchronous overlap as described, and it also transfers faster even synchronously, because the DMA engine can read it directly without the staging copy. The cost is that pinned memory cannot be swapped, so allocating too much of it pressures the system's memory. The rule is to pin the buffers involved in the hot transfer path, the ones you copy repeatedly, and leave the rest pageable. For a streaming pipeline like Dorado, pinning the signal input buffers and the result output buffers is exactly the right scope.

Key Insight

Async transfer and pinned memory are a package deal. *cudaMemcpyAsync* on pageable memory is a lie: it silently runs synchronously and your overlap never happens. If your Nsight Systems timeline shows no overlap despite using streams and async copies, the first thing to check is whether the host buffers are pinned with *cudaMallocHost*. This single oversight defeats more stream-based optimizations than any other, precisely because it fails silently with no error, just a timeline that stubbornly refuses to overlap.

15.4 Timing GPU Work Correctly with *cudaEvent*

Before we go further, a correctness point about measurement that matters for every number you report. You cannot time GPU work with the CPU clock the way you timed CPU work in the benchmark programs. The reason is that kernel launches are asynchronous: the CPU issues the launch and returns immediately, before the kernel has even started running on the GPU. If you wrap a kernel launch in *clock_gettime*, you measure the launch overhead, a microsecond or two, not the kernel's actual execution time, which might be milliseconds. The CPU clock is measuring the wrong thing entirely.

The correct way is CUDA events, which are timestamps recorded on the GPU's own timeline. You record an event before the work, record another after, synchronize, and ask for the elapsed time between them. This measures the actual GPU execution, because the events sit in the GPU's stream and are timestamped when the GPU reaches them.


```

cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);           // timestamp on the GPU timeline
my_kernel<<<grid, block>>>(...); // async launch
cudaEventRecord(stop);           // timestamp after the kernel
cudaEventSynchronize(stop);      // wait until the GPU reaches 'stop'

float ms;
cudaEventElapsedTime(&ms, start, stop); // true GPU time, in ms
printf("kernel: %.3f ms\n", ms);

cudaEventDestroy(start);
cudaEventDestroy(stop);

```

C

This is exactly the pattern the `matmul_cuda` benchmark uses to time its two kernels, and now you know why it uses `cudaEvent` rather than the `clock_gettime` that the CPU benchmarks used. The `cudaEventSynchronize` is essential: it blocks the CPU until the GPU has actually reached the stop event, so that `cudaEventElapsedTime` reports a completed measurement rather than a partial one. The elapsed time is in milliseconds with sub-microsecond resolution, measured on the GPU's clock, which is the truth of how long the kernel ran. Use this pattern whenever you time GPU work in your own code; trust Nsight Systems and Nsight Compute for the deeper analysis, but `cudaEvent` is the right tool for a quick, correct timing in your own instrumentation.

Common Pitfall

Timing a kernel launch with `clock_gettime` or `std::chrono` on the CPU. The launch is asynchronous and returns before the kernel runs, so you measure launch overhead (microseconds), not execution (milliseconds), and conclude your kernel is absurdly fast. Always use `cudaEvent` for GPU timing, and always `cudaEventSynchronize` before reading the elapsed time. This mistake produces the most confidently wrong GPU benchmark numbers in existence.

15.5 Seeing the Overlap in Nsight Systems

The whole point of streams is overlap, and the only way to confirm you got it is the Nsight Systems timeline. Profile the streamed version and compare it to the serial version, both with NVTX annotations marking the chunks.

```

nsys profile --trace=cuda,nvtx --output=streamed ./matmul_streamed
# open streamed.nsys-rep and look at the stream rows

```

bash

In the serial version, you saw one row of activity with transfer and kernel bars strictly sequential, the GPU idle during transfers. In the streamed version with pinned memory, you see multiple stream rows with their bars staggered: while stream 0's kernel runs, stream 1's transfer runs alongside it, and the GPU compute row stays busy across what used to be transfer gaps. You can also compare

the pageable-versus-pinned case directly: profile once with malloc'd host buffers and once with cudaMallocHost buffers, and the timeline will show the pageable version stubbornly serial (the silent synchronous fallback) and the pinned version properly overlapped. That side-by-side is the most convincing demonstration of why pinned memory matters, and it is exactly the kind of before-and-after the report benefits from.

15.6 Multi-GPU, Briefly

When one GPU is not enough, you distribute work across several. The basic mechanism is `cudaSetDevice` to select which GPU subsequent operations target, combined with one set of streams and buffers per GPU. You partition the work, assign a partition to each GPU, and let them run concurrently. Nsight Systems profiles all GPUs at once, showing a set of timeline rows per device, so you can see whether the work is balanced across them or whether one GPU finishes early and idles while another is still busy. Imbalance across GPUs is the multi-GPU analogue of the load imbalance you watch for across CPU cores, and the timeline reveals it the same way.

For the Nanopore project this matters on the LUNA cluster, which has multiple GPUs per node. Dorado can use several GPUs to basecall in parallel, and profiling the multi-GPU run with Nsight Systems tells you whether the work distributes evenly and whether the GPUs are kept busy or are stalling on a shared bottleneck like disk I/O feeding the signal data. Multi-GPU scaling is its own deep topic, but the profiling principle is the same as everything in this batch: look at the timeline, find the idle gaps, and close them, now across several devices instead of one.

How This Connects to Your Project

Dorado is fundamentally a streaming pipeline: raw signal chunks flow in, get basecalled, and bases flow out, over and over. This is precisely the workload that streams and pinned memory transform. If the Nsight Systems timeline from Chapter 12 showed the GPU idle while signal data was transferred in, the fix is exactly this chapter: overlap the transfer of the next signal chunk with the basecalling of the current one, using multiple streams and pinned host buffers for the signal data.

The report can include a host-side analysis alongside the kernel analysis: "the Nsight Systems timeline shows the GPU idle X percent of the time during signal transfer; overlapping transfers with computation via streams and pinned memory would close this gap." That recommendation addresses a bottleneck that no in-kernel optimization could touch, and it complements the in-kernel roofline analysis from Chapter 13. Together they cover both halves of GPU performance: how busy each kernel is internally, and how well the kernels are fed externally. The LUNA cluster's multiple GPUs extend this to the question of whether parallel basecalling balances the work, which the multi-GPU timeline answers directly.

What We Just Learned

The default CUDA stream serializes everything, so transfers and kernels never overlap and the GPU sits idle during transfers. The fix lives between kernels, not inside them.

CUDA streams are ordered queues; operations in different streams can overlap. Double-buffering across two streams lets the transfer of the next chunk hide behind the computation of the current one, closing the GPU-idle gaps.

Async overlap requires pinned (page-locked) host memory via `cudaMallocHost`. `cudaMemcpyAsync` on ordinary pageable memory silently falls back to a blocking copy, defeating overlap with no error. This silent failure defeats more stream optimizations than anything else.

Time GPU work with `cudaEvent`, not the CPU clock. Kernel launches are asynchronous and return before the kernel runs, so CPU timing measures launch overhead. Record start and stop events, `cudaEventSynchronize`, then `cudaEventElapsedTime`. This is why `matmul_cuda` uses events.

Confirm overlap in the Nsight Systems timeline: serial runs show sequential bars with idle gaps; streamed runs with pinned memory show staggered, overlapping stream rows with the GPU busy throughout. The pageable-versus-pinned comparison is the clearest demonstration of why pinning matters.

Multi-GPU work uses `cudaSetDevice` with per-device streams and buffers; Nsight Systems profiles all devices and reveals load imbalance across them, the multi-GPU analogue of CPU-core imbalance. On LUNA this is how you check that parallel Dorado basecalling balances the work.

Before You Move On

You have now completed the GPU batch and the entire Nsight toolkit. You can survey a GPU application's timeline with Nsight Systems, dissect any kernel with Nsight Compute, classify it on the roofline, act on its occupancy and bank conflicts and warp stalls, and close the host-side gaps with streams and pinned memory. Combined with Batches 1 and 2, you can now profile any program on either side of the PCIe bus, from a single CPU cache line to a multi-GPU pipeline.

Batch 4 is the synthesis. It covers Intel VTune and its Top-Down Methodology, the Linux system tools (`time`, `strace`, `ltrace`, `vmstat`, `numactl`) that frame every investigation, LIKWID for hardware-counter expertise and memory-bandwidth measurement, and then the two chapters everything has been building toward: how to actually write the two-page profile report for Prof. Paul, and the decision framework for choosing the right tool for any future question. The tools are nearly all in your hands now; Batch 4 is about wielding them as a coherent practice and producing the deliverable.